

SPRING

Introduction

Framework:

- When you develop an application, there are lot of repeatable codes (common logics) goes into your project.
- Framework has ability to generate these common logics itself dynamically. This makes programmer to develop only application specific logics.

Types of Framework—

- Web application framework—spring mvc
- Orm framework
- Application Development framework—Spring Framework
- Distrubuted Application Development Framework.
 - Eg. Gpay → Amazon → Customer
 - Here Gpay is Distributed app.
- Big-data framework- u tube, google generates big data, these data cannot be stored in regular database softwares.
- Based on mode of application we have—
 - 1] Invasive Framework – Servlet—
 - 2] Non-Invasive Framework -- Spring comes in this category

Spring:

- The Spring Framework is a powerful, and highly flexible framework focused on building web applications in Java.
- It developed by Rod Johnson in 2003.
- Spring is Non-Invasive framework it means no need to extend any class or any interface.
- Spring makes programming Java quicker, easier, and safer for everybody. It's focus on speed, simplicity, and productivity has made it the world's most popular Java framework.
- We can easily integrate with any framework using spring framework so it is framework of framework.
- Spring provides container concepts will manage our java classes.

Features of Spring Framework:

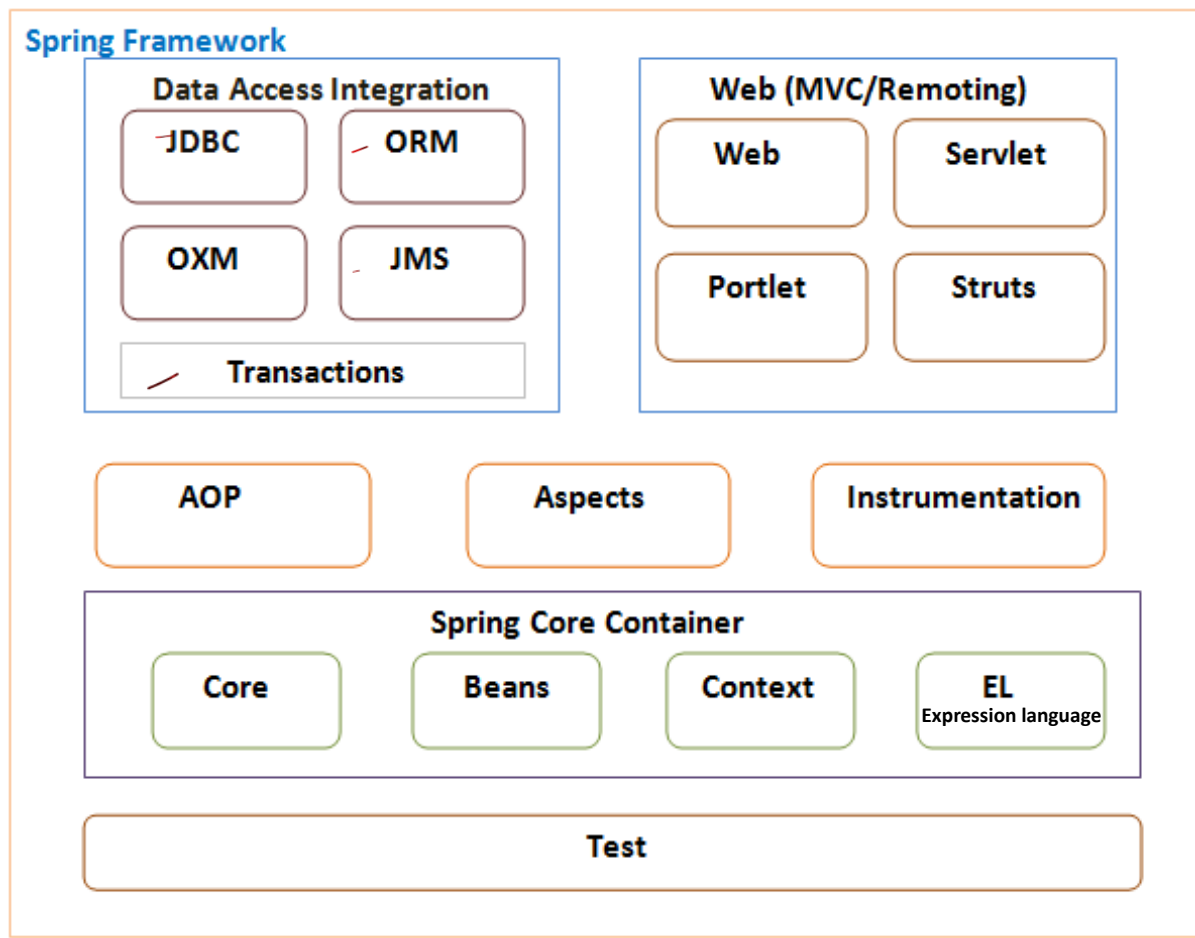
Here are some key features of spring framework:

- Inversion of Control (IoC)
- Dependency Injection (DI)
- Aspect-Oriented Programming (AOP)
- Spring MVC
- Data Access
- Transaction Management
- Spring Boot
- Spring Security
- Spring Data
- Spring Testing
- Spring RESTful Web Services
- Spring Cloud

Advantages Of Spring Framework:

- Spring is **non-invasive** : as it does not forces us to extend any class or implement any interface
- Spring is light weight: As It supports POJO model
- Loose coupling: As we do not need to change java code
- Spring has its own container, so it does not need server to run application
- Spring supports all (Web, Mobile based, Desktop based) applications
- Spring can integrate another framework/tool easily

Spring Modules:



Spring Core:

- Spring Core solves big issue in a software world that is dependencies (Object).
- Spring Core is the heart of entire Spring. contains some base framework classes, principles, and mechanisms.
- The entire Spring Framework and other projects of Spring are developed on top of the Spring Core.
- The Spring Contains Following Important Components- IOC (Inversion of Control).
- Eg. In simple words, if u need a hard drive so u do not need to create object of Hard drive it will be done and it will be injected in your application automatically.

IOC: Inversion Of Control-

- Inversion of control is a principle which helps to delegate the control.
- To apply this principle, spring framework has its own IOC Container.
- So, Spring have IOC container that can create objects for you.

How to use IOC Container:

- Spring Framework gives you two types of IOC container
 1. BeanFactory (Interface)(Core Container)
 2. ApplicationContext (Interface) (J2EE Container)

1. Core Container-

- It is Interface
- With the help of core container we can develop only desktop based application.
- **Way of use:**
 Resource `rs` = `new` ClassPathResource(`spring configuration file path`);
 BeanFactory `bean` = `new` XmlBeanFactory(`rs`);

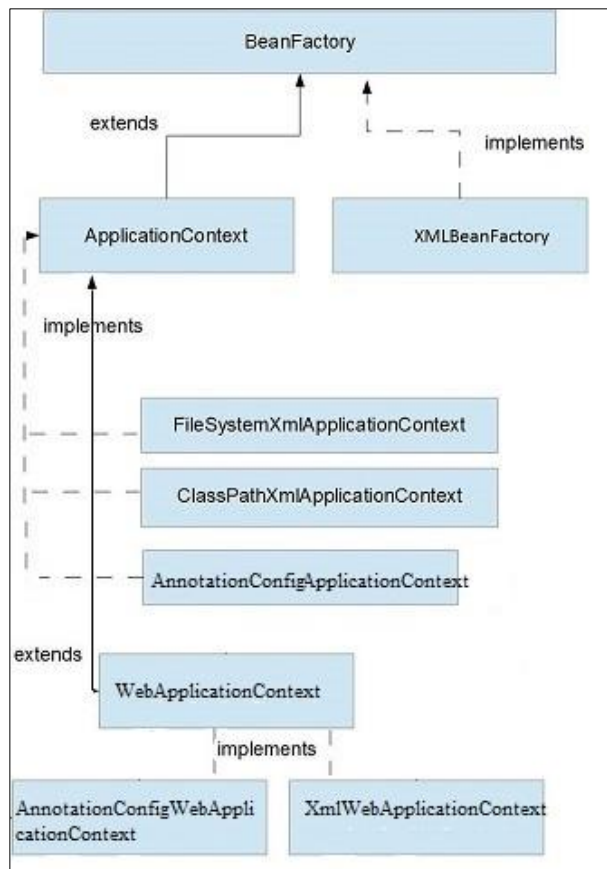
2. Application Context

- It provides Advanced Features to us. So, we are going to use this.
- With the help of J2EE container we can develop only desktop based as well as web based application.
- So it is better than bean factory.
- It has three Implementation classes:
 - o ClassPathXmlApplicationContext – We are going to use this because it has capability to read xml file from source folder.
 - o AnnotationConfigApplicationContext
 - o FileSystemXMLApplicationContext
- **Way of use:**

ApplicationContext `context` = `new` ClassPathXmlApplicationContext(`spring configuration file path`);

- **XML doctype—**
- <https://docs.spring.io/spring-framework/docs/4.2.x/spring-framework-reference/html/xsd-configuration.html>

Implementation of BeanFactory and ApplicationContext Interface—



Key- Features of Container:

1. It Creates instance of pojo class.
2. To manage life cycle of pojo class.
3. Dependency injection (D.I) of pojo class.

Difference Between Beafactory and ApplicationContext:

<u>BeanFactory</u>	<u>ApplicationContext</u>
1. It is a fundamental container that provides the basic functionality for managing beans.	It is an advanced container that extends the BeanFactory that provides all basic functionality and adds some advanced features.
2. It is suitable to build standalone applications.	It is suitable to build Web applications.

3. It does not support Annotations. In Bean Auto wiring, we need to configure the properties in XML file only.	It supports Annotation based configuration in Bean Auto wiring.
4. Post resource should add manually By using Resource r = new classpathResource(" bean.xml"); BeanFactory bf= new XMLBeanFactory(r);	Post resource will add automatically
5. It does not support Internationalization	It does not support Internationalization

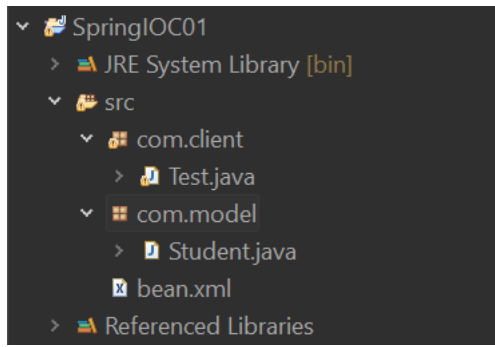
Some jar Files Required for Spring IOC:

1. Spring AOP
2. Spring Aspect
3. Spring Beans
4. Spring Context
5. Spring Context indexer
6. Spring Context support
7. Spring Core
8. Spring Expression
9. Spring Instrument
10. Spring Jcl

Example-1

XML Based--

Folder Structure-



Steps:

1. Create a simple java project and add above jar files in build path.
2. Create a model class as below.

Student.java

```
package com.model;

public class Student {

    public Student() {

        System.out.println("Student Class Constructor Called..!");

    }

}
```

3. Create a xml file in src folder.
[This is boiler plate code available of xml-based-configuration on spring-official-documentation]

Bean.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
```

```
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="stu" class="com.model.Student"></bean>  
</beans>
```

4. Create a Test class using Core Container- [If you want to use Core Container]

Test.java

```
package com.client;  
  
import org.springframework.beans.factory.BeanFactory;  
import org.springframework.beans.factory.xml.XmlBeanFactory;  
import org.springframework.core.io.ClassPathResource;  
import org.springframework.core.io.Resource;  
  
@SuppressWarnings("deprecation")  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        Resource r = new ClassPathResource("bean.xml");  
        BeanFactory bean = new XmlBeanFactory(r);  
        Student s = (Student) bean.getBean("stu");  
    }  
}
```

5. Create a Test class using Application Container. - [If you want to use J2EE Container]

Test.java

```
package com.client;  
  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
public class Test {
```



```
public static void main(String[] args) {  
    ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");  
        context.getBean("stu");  
    }  
}
```

Spring bean life cycle :

- spring bean life cycle specifies journey of bean starting from bean creation and bean distruction.
- In Spring bean life cycle there, some steps are performerd:
Application Started → Container Creation → All Singleton scope is loaded in container
→ All dependencies are injected completed → If Custom init method is present then it will call → other methods of application are called → Container is closed → At the time of containser closing if custom destroy method is present then it will call.
- **Note:** To Configure init and destroy we need to add init-method and destroy-method attributes in bean tag.
Eg. `<bean id="vp" class="VPCollege" init-method="init" destroy-method="destroy"/>`

Example-

VPCollege.java

```
public class VPCollege {  
    public void init() {  
        System.out.println("Init-method called..!");  
    }  
  
    VPCollege()  
    {  
        System.out.println("VP::Constructor Called..!");  
    }  
  
    public void admisssion()  
    {  
        System.out.println("VP:: Admission Called..!");  
    }  
}
```

```

    public void Passout()
    {
        System.out.println("VP:: Student Passout..!");
    }

    public void destroy() {
        System.out.println("Destroy-method called..!");
    }
}

```

Bean.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="vp" class="VPCollege" init-method="init" destroy-method="destroy"/>
</beans>

```

Test.java

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.ConfigurableApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {

```

```

        System.out.println("Main--starts..");
ApplicationContext context = new ClassPathXmlApplicationContext("bean.xml");
        VPCollege vp = (VPCollege) context.getBean(VPCollege.class);

        vp.admisssion();
        vp.Passout();

        ((ConfigurableApplicationContext)context). close();
        System.out.println("Main-end");
    }
}

```

Scope of Bean :

- Bean scope refers the life cycle of bean(object) means, when the object of bean will be instantiated, how long does that object live, and how many object will be created for that Bean through.
- Scope Controls the instance creation of the bean and it is managed by spring container.
- To change the scopes of bean we need to add attribute in bean tag.
Eg. <bean id= "stu" class= "com.model.student" scope= "prototype"/>
- There are five types of beans:

- I. For Desktop Based Applications-
 - i. Singleton ..bydefault
 - ii. Prototype
- II. For Web Based Applications-
 - i. Request
 - ii. Session
 - iii. Application
 - iv. Websocket

Singleton: -

- It is use for **single** object (instance) in one project.

- Only one instance will be created for a single bean definition per spring IOC container and the same object will be shared for each request made for that bean.
- **by default** scope of every bean is **singleton**.

Prototype: -

- new bean created every time when getBean method is called.
- ApplicationContext will act a lazy loading for perticular bean when spring bean scope is prototype.

Note-

1. **By default, spring framework use singleton design pattern.**
2. **why spring keeps bean scope is singleton by default?**
 - In our project maximum time we need only single instance of class to call the methods.
 - it will save memory heads and increase memory efficiency of our project.

Note-

1. **When Scope = singleton**
 - a. Bean Factory is works as a lazy loading.
 - b. ApplicationContext works as a Eager Loading
2. **When Scope = Prototype**
 - a. Bean Facory is works as a lazy loading.
 - b. ApplicationContext is works as a lazy loading.

Lazy and Eager Loading Concept:

1. Lazy Loading: -

- Lazy loading is the practice of delaying load or initialization of resources or objects until they are actually needed to improve performance and save system resources.
- For example- If a web page has an image that the user has to scroll down to see, you can display a place holder and lazy load the full image only when the user arrives to its location.

2. Eager Loading: -

- Eager loading initializes or loads a resource as soon as the code is executed. Eager loading is beneficial when there is an opportunity or need to load resource in the background.
- For example-Some websites display a loading screen and eagerly load all the resources required for the web application.

Example:

Bean.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="vp" class="VPCollege" scope="prototype"/>
    <bean id="tc" class="TCCollege" scope="singleton"/>
</beans>
```

TCCollege.java

```
public class TCCollege {
    public TCCollege() {
        System.out.println("Tc College :: Constructor Called..!");
    }
}
```

VPCollege.java

```
public class VPCollege {

    VPCollege()
    {
```

```
        System.out.println("VP::Constructor Called..!");  
    }  
}
```

Test.java

```
import org.springframework.context.ApplicationContext;  
import org.springframework.context.ConfigurableApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
  
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Main--starts..");  
  
        ApplicationContext context = new  
        ClassPathXmlApplicationContext("bean.xml");  
  
        VPCollege vp = (VPCollege) context.getBean(VPCollege.class);  
  
        System.out.println("Main-end");  
    }  
}
```

Dependency Injection :

- Dependencies are nothing but out data members and assigning values to them is a Dependency Injection
- The Dependency Injection is a design pattern that removes the dependency of the programs. In such case we provide the information **from the external source** such as **XML file**. It makes our code **loosely coupled and easier for testing**.
- The Dependency Injection can be achieved in two ways:
 1. Setter Injection
 2. Constructor Injection

1. Setter Injection:

- In setter injection, IOC container calls the setter method of target class to assign dependent class object to target class object.

- Dependency injection makes our program code loosely coupled. And easier for testing. We provide information from the external source such as xml file
- **Note:** In Setter Injection, container uses [setter methods](#) in POJO class to inject dependencies.
- In this, our class must contain setter methods otherwise we cannot inject dependencies.

Example:

1] Primitive Type/ String Based Values:

- In this simple employee class is there and we inject values using xml file.

Student.java

```
public class Student {  
  
    private int rno;  
  
    private String name;  
  
    private float marks;  
  
    public int getRno() {  
        return rno;  
    }  
  
    public void setRno(int rno) {  
        this.rno = rno;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public float getMarks() {  
        return marks;  
    }  
  
    public void setMarks(float marks) {
```



```

        this.marks = marks;
    }

    @Override
    public String toString() {
        return "Student [rno=" + rno + ", name=" + name + ", marks=" + marks + "];"
    }
}

```

beans.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="stu" class="Student">
        <property name="rno" value="101" />
        <property name="name" value="Rushi" />
        <property name="marks" value="88" />
    </bean>
</beans>

```

Test.java

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new
        ClassPathXmlApplicationContext("beans.xml");
    }
}

```

```

Student stu = (Student)context.getBean("stu");

System.out.println("Roll No:"+stu.getRno());
System.out.println("Name:"+stu.getName());
System.out.println("Marks:"+stu.getMarks());

System.out.println(stu.toString()); //using toString
    }
}

```

2] Setter Injection with Dependent Object:

- If there is HAS-A relationship between the classes, we create the instance of dependent object (contained object) first.

Example: **Student.java**

```

public class Student {

    int sid;

    String name;

    private Address address;w

    public int getSid() {
        return sid;
    }

    public void setSid(int sid) {
        this.sid = sid;
    }
}

```

```

    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public Address getAddress() {
        return address;
    }

    public void setAddress(Address address) {
        this.address = address;
    }

    @Override
    public String toString() {
        return "Student [sid=" + sid + ", name=" + name + ", address=" + address +
    "];
    }
}

```

Address.java

```

public class Address {
    String city;

```

```
String country;
```

```
public String getCity() {  
    return city;  
}
```

```
public void setCity(String city) {  
    this.city = city;  
}
```

```
public String getCountry() {  
    return country;  
}
```

```
public void setCountry(String country) {  
    this.country = country;  
}
```

```
@Override
```

```
public String toString() {  
    return "Address [city=" + city + ", country=" + country + "];"  
}
```

```
}
```

Beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans  
  xmlns="http://www.springframework.org/schema/beans"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-  
instance"
```

```

    xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-
    beans.xsd">

    <bean id="stu" class="Student">
        <property name="sid" value="101" />
        <property name="name" value="Rushi" />
        <property name="address"
ref="add"></property>
    </bean>

    <bean id="add" class="Address">
        <property name="city" value="Baramati"/>
        <property name="country" value="Pune"></property>
    </bean>

</beans>

```

Test.java

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new
        ClassPathXmlApplicationContext("beans.xml");

        Student stu =(Student)context.getBean("stu");

        System.out.println(stu);

    }
}

```

```
}
```

Constructor Injection:

- In Constructor Injection, container uses **constructor** in class to inject dependencies.

Example:

Primitive type:

Student.java

```
public class Student {  
  
    private int rno;  
    private String name;  
    private float marks;  
    public Student(int rno, String name, float marks) {  
        super();  
        this.rno = rno;  
        this.name = name;  
        this.marks = marks;  
    }  
  
    @Override  
    public String toString() {  
        return "Student [rno=" + rno + ", name=" + name + ", marks=" + marks + "];"  
    }  
}
```

beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean id="stu" class="Student">
    <constructor-arg value="101" />
    <constructor-arg value="Rushi" />
    <constructor-arg value="88" />
</bean>
</beans>
```

Test.java

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {
    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");

        Student stu = (Student) context.getBean("stu");

        System.out.println(stu.toString());
    }
}
```

Example:

- If there is HAS-A relationship between the classes, we create the instance of dependent object (contained object) first then pass it as an argument of the main class constructor.

Student.java

```
public class Student {  
  
    int sid;  
  
    String name;  
  
    private Address address;  
  
    public Student(int sid, String name, Address address) {  
        super();  
        this.sid = sid;  
        this.name = name;  
        this.address = address;  
    }  
  
    @Override  
    public String toString() {  
        return "Student [sid=" + sid + ", name=" + name + ", address=" + address +  
        "];"  
    }  
}
```

Address.java

```
public class Address {  
  
    String city;  
  
    String country;  
  
    public Address(String city, String country) {  
        super();  
        this.city = city;  
    }  
}
```



```

        this.country = country;
    }

    @Override
    public String toString() {
        return "Address [city=" + city + ", country=" + country + "]";
    }
}

```

beans.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
    xsi:schemaLocation="
    http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-
beans.xsd">

    <bean id="stu" class="Student">
        <constructor-arg value="101" />
        <constructor-arg value="Rushi" />
        <constructor-arg ref="add"/>
    </bean>

    <bean id="add" class="Address">
        <constructor-arg value="Pune" />
        <constructor-arg value="Country" />
    </bean>

</beans>

```

Test.java

```

import org.springframework.context.ApplicationContext;

```

```

import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");

        Student stu =(Student)context.getBean("stu");

        System.out.println(stu);

    }

}

```

Difference Between Setter Injection and Constructor Injection

<u>Setter Injection</u>	<u>Constructor Injection</u>
1. Partial injection of dependancies is possible.	1. Partial injection of dependancies is not possible.
2. Setter injection makes bean class object as mutable (changeable).	3. Constructor injection makes bean class object as immutable (unchangeable).

Dependency Injection for Collection Type:

- Spring also supports dependency injection for collection types, such as lists, sets, and maps.
- This feature allows you to inject a collection of objects into a bean, making it useful for scenarios where you need to manage multiple instances of dependencies.

Example:

Emp.java

```

import java.util.List;

import java.util.Map;

```

```
import java.util.Set;
```

```
public class Emp {
```

```
    private String name;
```

```
    private List<String> phones;
```

```
    private Set<String> address;
```

```
    private Map<String, String> courses;
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public void setName(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    public List<String> getPhones() {
```

```
        return phones;
```

```
    }
```

```
    public void setPhones(List<String> phones) {
```

```
        this.phones = phones;
```

```
    }
```

```
    public Set<String> getAddress() {
```

```
        return address;
    }
```

```
    public void setAddress(Set<String> address) {
        this.address = address;
    }
```

```
    public Map<String, String> getCourses() {
        return courses;
    }
```

```
    public void setCourses(Map<String, String> courses) {
        this.courses = courses;
    }
```

```
    public Emp(String name, List<String> phones, Set<String> address,
Map<String, String> courses) {
        super();
        this.name = name;
        this.phones = phones;
        this.address = address;
        this.courses = courses;
    }
```

```
    public Emp() {
        super();
    }
```

```

        @Override

        public String toString() {

            return "Name=" + name + ", Phones=" + phones + "\n Address=" +
address + "\n Courses=" + courses;

        }

    }

```

Test.java

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

```

```

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new
ClassPathXmlApplicationContext("beans.xml");

        Emp e1 = (Emp) context.getBean("emp1");

        // System.out.println(e1.getName());
        // System.out.println(e1.getPhones());
        // System.out.println(e1.getAddress());
        // System.out.println(e1.getCourses());

        System.out.println(e1);

    }
}

```

beans.java

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="
```

```
    http://www.springframework.org/schema/beans
```

```
    http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
    <bean class="Emp" name="emp1">
```

```
        <property name="name" value="Tekno" /> <!-- the method we  
used is setter injection -->
```

```
        <property name="phones">
```

```
            <list>
```

```
                <value>9696969573</value>
```

```
                <value>9696969</value>
```

```
                <value>969573</value>
```

```
                <null />
```

```
            </list>
```

```
        </property>
```

```
        <property name="address">
```

```
            <list>
```

```
                <value>Pune</value>
```

```
                <value>PCMC</value>
```

```
                <value>Bmt</value>
```

```
            </list>
```

```
        </property>
```

```
<property name="courses">
    <map>
        <entry key="java" value=" 2-Month" />
        <entry key="Python" value=" 1.2 months" />
        <entry key="C" value="1months" />
    </map>
</property>

</bean>

</beans>
```

Autowiring:

- Auto wiring is a concept where there is no need to inject secondary dependencies. Spring container will inject secondary dependencies on its own.
- Auto wiring supports only secondary type dependencies (not primitive).
- By default, Autowiring is disabled. If we enable Autowiring, Spring container will take care of injecting secondary type dependencies.

Autowiring can be achieved –

1. byName
2. byType
3. Constructor
4. autoDetect..**[deprecated]**
5. none (bydefault)

Advantages of Autowiring-

- It is done Automatic.
- Less code

Dis-Advantages of Autowiring-

- There is no control of programmer.
- It can't be used on primitive and String Values.

1.byName:

- The by Name mode injects the object dependency according to name of the bean. In this case, bean id and reference name must be same.
- For e.g. Consider Student class with secondary dependency of Address class as 'address' then: -
- **Spring** container will find secondary dependency in Student class, then it will take name (add) & matches it with bean id.
- If match found, then it verifies the class of bean & type of secondary injection is same or not.

Example:

Student.java

```
public class Student {

    int sid;

    String name;

    private Address address;

    public void setAddress(Address address) {
        this.address = address;
    }

    public void setSid(int sid) {
        this.sid = sid;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Student [sid=" + sid + ", name=" + name + ", address=" + address + "];"
    }
}
```



```
}
```

Address.java

```
public class Address {  
  
    String city;  
  
    String country;  
  
    public String getCity() {  
        return city;  
    }  
  
    public void setCity(String city) {  
        this.city = city;  
    }  
  
    public String getCountry() {  
        return country;  
    }  
  
    public void setCountry(String country) {  
        this.country = country;  
    }  
  
    @Override  
    public String toString() {  
        return "Address [city=" + city + ", country=" + country + "];"  
    }  
  
}
```

Beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="  
http://www.springframework.org/schema/beans  
http://www.springframework.org/schema/beans/spring-beans.xsd">  
  
    <bean id="stu" class="Student" autowire="byName">  
        <property name="sid" value="101" />  
    </bean>  
</beans>
```

```

        <property name="name" value="Rushi" />
    </bean>

    <bean id="address" class="Address">
        <property name="city" value="Bmt" />
        <property name="country" value="India" />
    </bean>

</beans>

```

Test.java

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
        Student stu = (Student) context.getBean("stu");

        System.out.println(stu);
    }
}

```

2.byType:

- The byType mode injects the object dependency according to type. In this case, bean id and reference name may be different. But there must be only one bean of a type.
- For eg. Consider Student class with secondary dependency of Address class as 'address' then:-
- Spring container will find secondary dependency in Student class, then it will take type of that dependency (Address) & matches it with class in bean.
- If match found, it will inject property using Setter Injection.
- If match not found, property remains unwired.

Example:

Note:

All program is same as above program just change in beans.xml

beans.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="stu" class="Student" autowire="byType">
        <property name="sid" value="101" />
        <property name="name" value="Rushi" />
    </bean>

    <bean id="add" class="Address">
        <property name="city" value="Bmt" />
        <property name="country" value="India" />
    </bean>

</beans>

```

Note:

- If multiple bean with same class are found, ambiguity will occur. To resolve this add tag "autowire-candidate="false"", now that bean will not get involved in autowiring process.
- For example, If we have student and Employee class and both have address field with ref of Address class so we have to create Address class only one time because address class fields are same for both student and employee.
- So we create two beans of address class with two different address. but class name is same for both beans so if we go for ByType autowire then it throw exception([org.springframework.beans.factory.NoUniqueBeanDefinitionException](#): No qualifying bean of type 'com.cjc.Address' available: expected single matching bean but found 2: addr,eaddr)

- it found two beans with same class name. To resolve this add tag “autowire-candidate=“false””, now that bean will not get involved in autowiring process.

3.Constructor :

- The constructor mode injects the dependency by calling the constructor of the class. It calls the constructor having large number of parameters.
- It internally uses ‘byType’, but while injecting property it uses constructor based injection.

Example:

Student.java

```
public class Student {

    private int id;

    private String name;

    private float marks;

    private Address address;

    public Student(int id, String name, float marks, Address address) {
        super();
        this.id = id;
        this.name = name;
        this.marks = marks;
        this.address = address;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", marks=" + marks + ", address=" + address + "];"
    }

}
```

Address.java

```
public class Address {
```

```

private String city;
private String state;
private String Country;

public Address(String city, String state, String country) {
    super();
    this.city = city;
    this.state = state;
    Country = country;
}

@Override
public String toString() {
    return "city=" + city + ", state=" + state + ", Country=" + Country ;
}

}

```

beans.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="
http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="address" class="Address">
        <constructor-arg value="Pune"/>
    
```

```

        <constructor-arg value="MH" />
        <constructor-arg value="India" />
    </bean>

    <bean id="stu" class="Student"
autowire="constructor">
        <constructor-arg value="101"
type="int"/>
        <constructor-arg value="Rushi"/>
        <constructor-arg value="99.8"
type="float"/>
    </bean>
</beans>

```

Test.java

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
        Student stu = (Student) context.getBean("stu");
        System.out.println(stu);
    }
}

```

@Autowired Annotation:

- This annotation is applied to fields, setter methods, and constructors.
- The **@Autowired** annotation injects object dependency implicitly.

- When you use **@Autowired** on fields and pass the values for the fields using the property name, Spring will automatically assign the fields with the passed values.
- For @Autowired annotation to work, we also need to enable annotation based configuration in spring bean configuration file.
- This can be done **<context:annotation-config/>** **this element.**
or by defining a bean of type
org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor.

Example :

Student.java

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
public class Student {  
  
    private int id;  
  
    private String name;  
  
    private float marks;  
  
    @Autowired  
    private Address address;  
  
    public void setId(int id) {  
        this.id = id;  
    }  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
public void setMarks(float marks) {  
    this.marks = marks;  
}
```

```
public void setAddress(Address address) {  
    this.address = address;  
}
```

```
@Override
```

```
public String toString() {  
    return "Student [id=" + id + ", name=" + name + ", marks=" +  
marks + ", address=" + address + "];"  
}  
  
}
```

Address.java

```
public class Address {
```

```
    private String city;
```

```
    private String Country;
```



```

    public void setCity(String city) {
        this.city = city;
    }

    public void setCountry(String country) {
        Country = country;
    }

    @Override
    public String toString() {
        return "Address [city=" + city + ", Country=" + Country + "];"
    }
}

```

Test.java

```

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new
        ClassPathXmlApplicationContext("beans.xml");

        Student stu = (Student) context.getBean("stu");

        System.out.println(stu);
    }
}

```

by defining a bean of type

org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor.

beans.java

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<beans xmlns="http://www.springframework.org/schema/beans"
```

```
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
    xsi:schemaLocation="
```

```
        http://www.springframework.org/schema/beans
```

```
        http://www.springframework.org/schema/beans/spring-beans.xsd">
```

```
<bean
```

```
    class="org.springframework.beans.factory.annotation.AutowiredAnnotationBeanPostProcessor"/>
```

```
    <bean id="stu" class="Student">
```

```
        <property name="id" value="101"/>
```

```
        <property name="name" value="Rushi"/>
```

```
        <property name="marks" value="88"/>
```

```
    </bean>
```

```
    <bean id="address" class="Address">
```

```
        <property name="city" value="Pune"/>
```

```
        <property name="country" value="India"/>
```

```
    </bean>
```

```
</beans>
```

Done by **<context:annotation-config/>** this element.

- In this we need to add some extra boiler plate code.

beans.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-
                           beans.xsd
                           http://www.springframework.org/schema/context
                           http://www.springframework.org/schema/context/spring-
                           context.xsd">

    <context:annotation-config/>

    <bean id="stu" class="Student">
        <property name="id" value="101"/>
        <property name="name" value="Rushi"/>
        <property name="marks" value="88"/>
    </bean>

    <bean id="address" class="Address">
        <property name="city" value="Pune"/>
    </bean>
</beans>
```

```

        <property name="country" value="India"/>

    </bean>

</beans>

```

Use of @Qualifier annotation:

- The @Qualifier annotation is used to resolve the autowiring conflict, when there are multiple beans of same type.
- The @Qualifier annotation can be used on any class annotated with @Component or on method annotated with @Bean .
- This annotation can also be applied on constructor arguments or method parameters.
- This annotation is only work when we used **<context:annotation-config/>**
- And for that we need a boiler plate code of spring context.
- I provided that below:

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd
                           http://www.springframework.org/schema/context
                           https://www.springframework.org/schema/context/spring-
context.xsd">

```

```

    <context:annotation-config/>

```

```

</beans>

```

Example:

Student.java

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;

```

```
public class Student {  
  
    private int id;  
  
    private String name;  
  
    private float marks;  
  
    @Autowired  
    @Qualifier("add")  
    private Address address;  
  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public void setMarks(float marks) {  
        this.marks = marks;  
    }  
}
```

```

    public void setAddress(Address address) {
        this.address = address;
    }

    @Override
    public String toString() {
        return "Student [id=" + id + ", name=" + name + ", marks=" +
marks + ", address=" + address + "]";
    }
}

```

Beans.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans
xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xsi:schemaLocation="http://www.springframework.org/schema/beans
    http://www.springframework.org/schema/beans/spring-beans.xsd
    http://www.springframework.org/schema/context

```

<https://www.springframework.org/schema/context/spring-context.xsd>>

```
<context:annotation-config/>

<bean id="stu" class="Student">
    <property name="id" value="101"/>
    <property name="name"
value="Rushi"/>
    <property name="marks" value="88"/>
</bean>

<bean name="addr" class="Address">
    <property name="city"
value="Pune"/>
    <property name="country"
value="India"/>
</bean>

    <bean name="add" class="Address">
    <property name="city"
value="PCMC"/>
    <property name="country"
value="India"/>
    </bean>

</beans>
```

Note- Address.java and Test.java keep same.

Java Based :

- JavaBased program means in this program we can not used .xml file we used java class in place of .xml file for bean creation of the pojo class.

Annotations:

@Configuration:

- This annotation is used on classes that define beans. @Configuration is an analog for an XML configuration file – it is configured using Java classes.
- A Java class annotated with @Configuration is a configuration by itself and will have methods to instantiate and configure the dependencies.

@Bean:

- This annotation is used at the method level. The @Bean annotation works with @Configuration to create Spring beans.
- As mentioned earlier, @Configuration will have methods to instantiate and configure dependencies. Such methods will be annotated with @Bean. The method annotated with this annotation works as the bean ID, and it creates and returns the actual bean.

Example:1

Student.java

```
public class Student {  
  
    public Student() {  
  
        System.out.println("Student Cons Called..!");  
  
    }  
  
}
```

AppConfig.java

```
import org.springframework.context.annotation.Bean;  
  
import org.springframework.context.annotation.Configuration;
```


@Configuration

```
public class AppConfig {  
  
    @Bean(name = "stu")  
  
    public Student s() {  
  
        return new Student();  
  
    }  
  
}
```

Test.java

```
import org.springframework.context.ApplicationContext;  
  
import  
org.springframework.context.annotation.AnnotationConfigApplicationContext;  
  
public class Test {  
  
    public static void main(String[] args) {  
  
        ApplicationContext context = new  
        AnnotationConfigApplicationContext(AppConfig.class);  
  
        Student stu=context.getBean("stu",Student.class);  
  
        System.out.println(stu);  
  
    }  
  
}
```

Example:2

Student.java

```
public class Student {
```

```
int rno;

String name;

float marks;

public int getRno() {
    return rno;
}

public void setRno(int rno) {
    this.rno = rno;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public float getMarks() {
    return marks;
}

public void setMarks(float marks) {
    this.marks = marks;
}

public Student() {
    System.out.println("Student Cons Called..!");
}
```

```

    }

    @Override

    public String toString() {

        return "Student [rno=" + rno + ", name=" + name + ", marks=" +
marks + "];"

    }

}

```

AppConfig.java

```

import org.springframework.context.annotation.Bean;

import org.springframework.context.annotation.Configuration;

```

```

@Configuration

```

```

public class AppConfig {

```

```

    @Bean(name = "stu")

```

```

    public Student s() {

```

```

        Student stu = new Student();

```

```

        stu.setRno(101);

```

```

        stu.setName("Rushi");

```

```

        stu.setMarks(99);

```

```

        return stu;

```

```

    }

```

```

}

```

Test.java

```

import org.springframework.context.ApplicationContext;

import
org.springframework.context.annotation.AnnotationConfigApplicationContext;


public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);

        Student stu = context.getBean("stu", Student.class);

        System.out.println(stu.getName());

        System.out.println(stu);

    }

}

```

Autowiring in JavaBased:

- We are going to use @Autowire Annotation.

Example: Setter Based

Student.java

```

import org.springframework.beans.factory.annotation.Autowired;

public class Student {

    private int rno;

    private String name;

    private float marks;

```

@Autowired

private Address address;

```
public Address getAddress() {  
    return address;  
}
```

```
public void setAddress(Address address) {  
    this.address = address;  
}
```

```
public int getRno() {  
    return rno;  
}
```

```
public void setRno(int rno) {  
    this.rno = rno;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public void setName(String name) {  
    this.name = name;  
}
```

```
}
```

```
public float getMarks() {  
    return marks;  
}
```

```
public void setMarks(float marks) {  
    this.marks = marks;  
}
```

```
public Student() {  
    System.out.println("Student :: Constructor Called..!");  
}
```

```
@Override  
public String toString() {  
    return "Student [rno=" + rno + ", name=" + name + ", marks=" +  
marks + ", address=" + address + "];"  
}  
}
```

Address.java

```
public class Address {  
  
    private String city;
```

```
private String country;
```

```
public Address() {
```

```
    System.out.println("Address :: constructor Called..!");
```

```
}
```

```
public String getCity() {
```

```
    return city;
```

```
}
```

```
public void setCity(String city) {
```

```
    this.city = city;
```

```
}
```

```
public String getCountry() {
```

```
    return country;
```

```
}
```

```
public void setCountry(String country) {
```

```
    this.country = country;
```

```
}
```

```
@Override
```

```
public String toString() {
```

```
    return "Address: \ncity=" + city + "\n country=" + country;
```

```
}
```

```
}
```

AppConfig.java

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.context.annotation.Configuration;
```

```
@Configuration
```

```
public class AppConfig {
```

```
    @Bean(name = "stu")
```

```
    public Student s() {
```

```
        Student stu = new Student();
```

```
        stu.setRno(101);
```

```
        stu.setName("Rushi");
```

```
        stu.setMarks(99);
```

```
        return stu;
```

```
    }
```

```
    @Bean(name = "add")
```

```
    public Address a() {
```

```
        Address add = new Address();
```

```
        add.setCity("Baramati");
```

```
        add.setCountry("Pune");
```

```
        return add;
```

```
    }
```

```
}
```


Test.java

```
import org.springframework.context.ApplicationContext;

import
org.springframework.context.annotation.AnnotationConfigApplicationContext;


public class Test {

    public static void main(String[] args) {

ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);

        Student stu = context.getBean("stu", Student.class);

        System.out.println(stu);

    }

}
```

Example: Constructor Based

Student.java

```
import org.springframework.beans.factory.annotation.Autowired;


public class Student {

    private int rno;

    private String name;

    private float marks;
```

@Autowired

private Address address;

public Student(int rno, String name, float marks) {

 super();

 this.rno = rno;

 this.name = name;

 this.marks = marks;

}

@Override

public String toString() {

 return "Student [rno=" + rno + ", name=" + name + ", marks=" + marks + ", address=" + address + "];"

}

}

Address.java

public class Address {

 private String city;

 private String country;

 public Address() {

```

        System.out.println("Address :: constructor Called..!");
    }

    public Address(String city, String country) {
        super();
        this.city = city;
        this.country = country;
    }

    @Override
    public String toString() {
        return "Address: \ncity=" + city + "\n country=" + country;
    }
}

```

AppConfig.java

```

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class AppConfig {

    @Bean(name = "stu")
    public Student s() {
        Student stu = new Student(101, "Rushi", 99);
    }
}

```

```

        return stu;
    }

    @Bean(name = "add")
    public Address a() {
        Address add = new Address("PCMC", "India");

        return add;
    }
}

```

Test.java

```

import org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Test {

    public static void main(String[] args) {

        ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);

        Student stu = context.getBean("stu", Student.class);

        System.out.println(stu);

    }
}

```

}